Torneo de Fútbol: arquitectura de microservicios

Diseño propuesto para migrar el monolito actual (frontend estático + Supabase llamado directo desde el navegador) a microservicios seguros, con Node.js + TypeScript. Fecha: 15/10/2026.

1. Punto de partida y problemas a resolver

- **Lógica y acceso a datos en el navegador.** `js/db.js` consulta Supabase directo con la anon key. Toda la seguridad depende de las políticas RLS.
- **Login frágil.** La contraseña se hashea en el cliente (`DB.hashPassword`) y el login consulta la tabla `usuarios` . Un hash calculado en el cliente se vuelve la contraseña real y no protege nada.
- **Reglas de negocio sin servidor.** Resultados, multas, tabla de posiciones y notificaciones se calculan o se escriben desde el cliente, así que cualquiera con la anon key puede intentar alterarlos.
- **Claves en `config.js` .** La anon key es pública por diseño, pero las claves de EmailJS deberían salir del navegador y vivir en un servicio.

2. Servicios propuestos

Hay 5 servicios más un gateway. Cada uno es dueño de su base de datos y nadie más la toca.

Servicio	Responsabilidad	Tablas actuales que absorbe
api-gateway	Punto único de entrada, validación del JWT, rate limiting, CORS, enrutamiento.	(ninguna)
auth-service	Registro, login, refresh tokens, recuperación de contraseña, roles (admin, delegado de equipo).	`usuarios`
teams-service	Equipos, escudos, grupos, jugadores, validación de documento único.	`equipos` , `jugadores`
matches-service	Partidos, calendario, resultados, eventos del partido (goles, tarjetas), tabla de posiciones, goleadores y porteros.	`partidos` , `eventos_partido`

Servicio	Responsabilidad	Tablas actuales que absorbe
fines-service	Multas por tarjetas, su estado de pago y reglas de sanción.	multas (hoy dentro de `db.js`)
notifications-service	Notificaciones internas y correos. Es el único que tiene las claves del proveedor de correo.	`notificaciones`

Cada servicio tendría estructura propia: `routes`, `services`, `repositories`, esquemas de validación con Zod y migraciones con una herramienta como Drizzle o Prisma.

3. Diagrama

```
Navegador (frontend estático en Vercel / Cloudflare Pages)
       |  HTTPS + JWT
       v
 +-------------+
 | api-gateway |  rate limit, CORS, valida JWT, enruta
 +------+------+
        | red privada (token de servicio / mTLS)
 +------+-----------+-----------+-------------+
 |                  |           |             |
 v                  v           v             v
auth-service   teams-service  matches-service  fines-service
 |                  |           |    |           |
 DB auth        DB teams     DB matches   |      DB fines
                                          |
                      eventos asíncronos (cola) ---> notifications-service -
-> DB notifs
```

Ejemplo de flujo asíncrono: `matches-service` publica `partido.finalizado` y `tarjeta.registrada`. `fines-service` crea la multa y publica `multa.creada`, y `notifications-service` avisa al delegado del equipo. Así el registro de un resultado no depende de que el correo funcione.

4. Comunicación entre servicios

- **Síncrona (HTTP interno):** para consultas que necesitan respuesta inmediata, por ejemplo validar que un equipo existe.
- **Asíncrona (eventos):** para efectos secundarios. Para empezar basta Redis Streams o una cola simple; NATS o RabbitMQ solo si el volumen lo justifica.

- **Patrón outbox:** cada servicio guarda el evento en su propia base dentro de la misma transacción y otro proceso lo publica, para no perder eventos.
- **Contratos:** cada servicio expone OpenAPI y se valida en CI para que un cambio no rompa a los demás.

5. Seguridad (capas)

Autenticación y sesiones

- Contraseñas con **Argon2id** en el servidor. Se elimina el hash en el cliente.
- JWT de acceso de vida corta (10 a 15 min) y refresh token rotativo, en cookie `HttpOnly; Secure; SameSite=Strict`.
- Bloqueo progresivo por intentos fallidos y verificación de correo. Opcional: 2FA para administradores.

Autorización

- Roles (`admin`, `delegado`) verificados en cada servicio, no solo en el gateway.
- Un delegado solo puede modificar su propio equipo (control de propiedad en `teams-service`).

Datos

- Ninguna base de datos expuesta a internet: solo su servicio se conecta, con un usuario de mínimos privilegios.
- Consultas parametrizadas (ORM), validación de toda entrada con Zod y sanitización de salida contra XSS.
- Secretos solo en variables de entorno del proveedor; nunca en el repositorio. Rotar las claves actuales de `config.js` al migrar.
- Copias de seguridad probadas y cifrado en tránsito (TLS).

Perímetro

- Helmet, CSP estricta (ya existe una buena base en `vercel.json`), HSTS, rate limiting por IP y por usuario.
- Límite de tamaño en subida de escudos y validación del tipo real de archivo.

Operación

- Logs estructurados con identificador de correlación entre servicios, y registro de auditoría de acciones administrativas.
- CI con `npm audit`, análisis estático (CodeQL o Semgrep), escaneo de imágenes y pruebas de seguridad con OWASP ZAP, que ya mencionas en `render.yaml`.
- Health checks (`/health`) y apagado ordenado de cada servicio.

6. Bases de datos para producción (opciones gratuitas)

Pediste una base por microservicio. Con 5 servicios necesitas 5 bases de datos separadas. Datos verificados hoy:

Opción	Plan gratis	Encaja para 5 servicios
Neon (Postgres)	100 proyectos, 1 GB por proyecto y 20 GB en total, 100 horas de cómputo por proyecto. La base se suspende tras 5 min sin uso y despierta sola. Al llegar al límite no borra datos: bloquea escrituras o suspende el cómputo.	**Sí, es la recomendada.** Un proyecto por servicio.
Supabase	Solo 2 proyectos y 500 MB cada uno. Pausa el proyecto tras una semana de inactividad y hay que reactivarlo a mano.	No para 5 bases. Sirve como máximo para 2 servicios, y la pausa no conviene en producción.
Render Postgres	1 GB, pero **expira a los 30 días** y luego se elimina.	No.
Railway	Solo $1 de crédito mensual compartido entre servicios y bases.	No alcanza.

Recomendación: una base **Neon** por servicio, con sus conexiones pasando por el pooler que Neon ofrece. Cada servicio solo se conecta a la suya. Tu volumen (un torneo local) cabe de sobra en 1 GB por servicio. La contrapartida es el arranque en frío de unos instantes cuando la base estuvo suspendida.

Para el broker de eventos, Upstash Redis tiene plan gratuito, pero no verifiqué sus límites actuales; conviene confirmarlos antes de decidir.

7. Dónde alojar los servicios (gratis o casi)

Opción	Qué ofrece	Comentario
Google Cloud Run	2M peticiones, 180.000 vCPU-segundos y 360.000 GB-segundos al mes gratis. Escala a cero.	**La mejor para varios servicios**, pero exige tarjeta de crédito. Un presupuesto no frena el gasto, así que hay que poner alertas y límites de instancias.

Opción	Qué ofrece	Comentario
Render (web)	750 horas al mes **por workspace**, 512 MB de RAM, se duerme tras 15 min y tarda cerca de 1 minuto en despertar. Sin tarjeta.	Las horas se comparten entre todos tus servicios. Sirve para pruebas, pero los despertares encadenados entre servicios se sienten lentos.
Fly.io	Sin plan gratuito para cuentas nuevas y exige tarjeta.	Descartada.
Frontend	Vercel (Hobby) o Cloudflare Pages para el sitio estático.	Se mantiene lo que ya tienes.

Recomendación: frontend en Vercel o Cloudflare Pages, servicios en Cloud Run, bases en Neon. Si no quieres dar tarjeta, usa Render para empezar, sabiendo que habrá esperas de arranque.

8. Un punto honesto sobre la complejidad

Para un torneo local, 5 servicios con 5 bases es más complejidad operativa de la que la carga requiere (despliegues, versiones, trazabilidad, arranques en frío encadenados). Si lo que buscas es **seguridad y robustez**, la mayor parte se logra con un backend bien hecho. Si lo que buscas es aprender o demostrar la arquitectura de microservicios, la propuesta de arriba es válida. Una versión intermedia sería agrupar `fines-service` dentro de `matches-service` y `notifications-service` junto al gateway, quedando 3 bases, y separarlos más adelante si hace falta.

9. Plan por fases

1. **Fundaciones:** monorepo (pnpm workspaces), plantilla común de servicio, Docker Compose para desarrollo, CI.
2. **auth-service + gateway:** login seguro, JWT y roles. Es lo que más reduce el riesgo actual.
3. **teams-service:** equipos y jugadores, con migración de datos desde Supabase.
4. **matches-service:** partidos, resultados, posiciones y estadísticas.
5. **fines-service + notifications-service** con eventos asíncronos.
6. **Adaptar el frontend:** reemplazar `js/db.js` por un cliente de la API y retirar Supabase y las claves del navegador.
7. **Endurecimiento final:** pruebas de seguridad con ZAP, pruebas de carga, copias de seguridad y runbook.

Fuentes

- Neon: planes y límites
- Límites del plan gratuito de Supabase
- Render: plataformas con plan gratuito real en 2026
- Comparativa de planes gratuitos de hosting (septiembre 2026)

10. Validación contra la documentación del proyecto (GESTOR)

Leí el informe `Sistema_gestion_torneos_final_AD.docx` y lo crucé con el diseño anterior. Estos son los hallazgos que **cambian o ajustan** la arquitectura.

Dimensión real del sistema. El informe proyecta entre 100 y 200 usuarios activos, entre 40 y 80 equipos por torneo y más de 100 partidos por temporada. Es una carga muy pequeña: 1 GB por base en Neon sobra con amplitud, y la separación en microservicios se justifica por seguridad, aprendizaje y orden, no por rendimiento.

Ajustes al diseño de servicios

- **`fines-service` pasa a llamarse `discipline-service`.** El informe habla de tarjetas, multas económicas y **suspensiones automáticas por acumulación**, que es más que multas. Escucha los eventos de tarjetas de `matches-service` y calcula la suspensión.
- **Archivos (escudos y fotos de jugadores).** El carnet digital usa foto, nombre, dorsal y posición, y el registro permite subir un escudo. Hoy se guardan en base64 dentro de la base. Propuesta: guardarlos en almacenamiento de objetos (en local, MinIO; en producción, Cloudflare R2 o similar, a verificar) con límite de tamaño y validación del tipo real de archivo, y que la base guarde solo la URL.
- **Vista pública sin login.** Fixture y tabla de posiciones deben ser accesibles sin autenticación. Se resuelve con endpoints de solo lectura en el gateway, con caché de unos segundos y límite de peticiones por IP, para no exponer ningún servicio interno.
- **Tiempo real.** El informe promete actualización en tiempo real al registrar un resultado (hoy WebSockets de Supabase). Propuesta: Server-Sent Events desde el gateway, alimentados por los eventos de `matches-service`.
- **Exportación a Excel** de la tabla de posiciones: va en `matches-service`, generada en el servidor.
- **Generación automática de liguillas y grupos:** vive en `matches-service`, que necesita saber en qué grupo está cada equipo.
- **Campos que faltan en mis tablas:** cancha del partido, municipio del equipo, dorsal y posición del jugador, temporada del torneo, y asistencias en las estadísticas.

Consecuencia clave de separar las bases de datos. En el modelo actual `partidos` tiene claves foráneas a `equipos`, y `jugadores` a `equipos`. Con una base por servicio **esas claves ya no pueden existir**. Hay que decidir cómo se resuelve:

- `matches-service` guarda solo los identificadores de equipo, más una **copia mínima** (nombre, escudo, grupo) que se actualiza por eventos `equipo.creado` y `equipo.actualizado`. Así la tabla de posiciones se calcula sin consultar a otro servicio en cada petición.
- `discipline-service` hace lo mismo con jugadores y equipos.
- Al eliminar un equipo, un evento `equipo.eliminado` dispara la limpieza en los demás servicios (borrado lógico recomendado, para no perder el historial de resultados).

Inconsistencias del informe que conviene aclarar antes de migrar

- **Rol del delegado:** el informe dice que es de solo lectura, pero el panel actual permite agregar y eliminar jugadores y la propia lista de funcionalidades incluye "agregar nuevos jugadores". Defino la regla como: el delegado gestiona **solo la plantilla de su equipo** y lee todo lo demás. Confírmame si es correcta.
- **Autenticación:** el informe afirma que se usa Supabase Auth, pero el código valida contra la tabla `usuarios` con SHA-256 hecho en el navegador. Para migrar a Argon2id no se puede convertir un hash SHA-256 en uno nuevo, así que la migración de usuarios exige **reasignar contraseña** (enlace por correo) o rehashear al primer login exitoso.
- **Hosting:** el informe menciona a la vez Render y Vercel como despliegue del frontend. Lo trataré como Vercel o Cloudflare Pages en producción.
- **Un usuario puede gestionar uno o más equipos** según el modelo de datos, pero el flujo de registro crea un equipo por usuario. Mantengo la relación uno a muchos en `teams-service`.

Datos personales (muy relevante en Colombia). Se guardan nombres, documentos de identidad y fotos de jugadores, y puede haber menores de edad. Esto cae bajo la Ley 1581 de 2012 (habeas data). Propuesta: registrar el consentimiento con fecha y versión de la política de privacidad (hoy solo hay una casilla), cifrar el número de documento en reposo, no mostrarlo nunca en la vista pública, y poder borrar o anonimizar a un jugador si lo solicita. Esto no es asesoría legal; conviene que lo revise quien corresponda en la universidad o en el torneo.

11. Plan para empezar en local

La idea es que todo el sistema corra en tu computador con un solo comando, y que pasar a producción sea cambiar variables de entorno.

Estructura del repositorio (monorepo con pnpm)

```
gestor/
  apps/
    web/                 frontend actual, adaptado
  services/
    gateway/
    auth/
    teams/
    matches/
    discipline/
    notifications/
  packages/
    shared/              tipos, esquemas Zod, cliente de eventos, logger
  infra/
    docker-compose.yml
    postgres-init/       crea una base y un usuario por servicio
  docs/
```

Docker Compose en local

- Un contenedor **PostgreSQL** con **una base de datos y un usuario distinto por servicio**, de modo que ningún servicio pueda leer la base de otro ni siquiera por error. Es el equivalente local de los 5 proyectos de Neon.
- **Redis** como broker de eventos (Streams).
- **MinIO** para escudos y fotos.
- **Mailpit** para capturar los correos de recuperación de contraseña sin enviar nada real, en lugar de EmailJS.
- Los 6 servicios y el frontend, con recarga automática al editar código.
- Cada servicio con su `.env.example`; los `.env` reales no entran al repositorio.

Orden de construcción y criterio de "listo" de cada paso

1. **Base del monorepo:** pnpm, TypeScript estricto, ESLint, Prettier, pruebas con Vitest, plantilla de servicio con `/health`, logs estructurados y errores uniformes. Listo cuando `docker compose up` levanta todo en verde.
2. **auth + gateway:** registro, login, refresh rotativo, recuperación de contraseña, roles. Listo con pruebas de intentos fallidos, expiración de tokens y acceso cruzado entre roles.
3. **teams:** equipos y jugadores, escudos y fotos, control de propiedad del delegado. Listo con pruebas de que un delegado no toca el equipo de otro.
4. **matches:** partidos, liguillas automáticas, resultados, posiciones, goleadores, Excel. Listo cuando la tabla de posiciones coincide con la que hoy calcula `DB.getStandings()` sobre los mismos datos.

5. **discipline + notifications:** tarjetas, multas, suspensiones y avisos por eventos. Listo con una prueba de extremo a extremo: registrar un resultado produce multa y notificación.
6. **Migración de datos** desde Supabase con un script repetible que se pueda ensayar varias veces antes del cambio real.
7. **Frontend:** reemplazar `js/db.js` por un cliente de la API, quitar Supabase y las claves de `config.js`.
8. **Pruebas de seguridad** (OWASP ZAP, `npm audit`, revisión de dependencias) y de carga con el tamaño proyectado.
9. **Producción:** Neon, Cloud Run y frontend estático, usando las mismas imágenes de Docker que en local.

Qué voy a necesitar de ti

- Confirmar la regla del delegado y la versión de 5 o de 3 bases (ya dejé la pregunta en el documento).
- Un volcado de datos de prueba de Supabase (sin datos personales reales) para probar la migración, o confirmar que se parte desde cero.
- Confirmar si el administrador es una sola cuenta o puede haber varios organizadores, porque el informe dice que no hay multitorneo simultáneo.

12. Catálogo oficial de microservicios

Esta sección es la definición final: **si algo contradice las secciones 2 y 3** (por ejemplo el nombre `fines-service`), manda esta. No incluye código.

12.1 Reglas que se cumplen siempre

1. **Cada servicio es dueño de sus datos.** Solo él lee y escribe en su base de datos. Nadie más se conecta a ella.
2. **Ningún servicio llama a otro servicio para responder una petición.** Cuando necesita datos ajenos (por ejemplo el nombre de un equipo), guarda una **copia mínima** que se mantiene al día por eventos. La única pieza que habla con todos es el gateway.
3. **Todo lo que viene del navegador entra por el `api-gateway`.** Los demás servicios no son accesibles desde internet.
4. **Los efectos secundarios viajan por eventos** (avisos, multas, suspensiones, correos), no por llamadas directas.
5. **El rol del usuario se verifica en cada servicio**, no solo en el gateway.

12.2 Nombres y convenciones

Servicio	Carpeta	Base de datos	Puerto local	Ruta pública en el gateway
`api-gateway`	`services/gateway`	(ninguna)	8080	`/api/...`
`auth-service`	`services/auth`	`auth_db`	3001	`/api/auth/...`
`teams-service`	`services/teams`	`teams_db`	3002	`/api/teams/...`
`matches-service`	`services/matches`	`matches_db`	3003	`/api/matches/...`
`discipline-service`	`services/discipline`	`discipline_db`	3004	`/api/discipline/...`
`notifications-service`	`services/notifications`	`notifications_db`	3005	`/api/notifications/...`

Los eventos se nombran `<dominio>.<hecho en pasado>`, por ejemplo `partido.programado` o `multa.creada`. Las vistas públicas (sin login) viven bajo `/api/public/...` y son solo de lectura.

12.3 Ficha de cada servicio

`api-gateway` — la puerta de entrada

- **Responsabilidad:** recibir todo el tráfico del frontend, validar el token de sesión, limitar peticiones por IP, aplicar CORS, y enrutar al servicio que corresponde. También sirve las vistas públicas con caché de pocos segundos y el canal de tiempo real (Server-Sent Events) para resultados.
- **Es dueño de:** nada. No tiene base de datos ni reglas de negocio.
- **No hace:** no calcula posiciones, no guarda usuarios, no decide permisos finos (eso lo hace cada servicio).
- **Quién lo llama:** el navegador.

`auth-service` — identidad y acceso

- **Responsabilidad:** registro de cuentas (los delegados se registran libremente; los administradores solo con un código de acceso entregado por el desarrollador), inicio y cierre de sesión, renovación de sesión, recuperación de contraseña, bloqueo por intentos fallidos, roles (`admin` y `delegado`) y registro del consentimiento de privacidad.
- **Es dueño de:** usuarios (correo, contraseña cifrada, rol, estado), sesiones activas, solicitudes de recuperación, consentimientos con fecha y versión de la política, y los códigos de acceso de administrador.
- **No hace:** no sabe nada de equipos, jugadores ni partidos. No envía correos: pide el envío a `notifications-service` por evento.

- **Publica:** `usuario.registrado`, `usuario.correo_actualizado`, `usuario.recuperacion_solicitada`, `usuario.desactivado`.
- **Consume:** nada.

`teams-service` — equipos y jugadores

- **Responsabilidad:** alta, edición y baja de equipos (nombre, municipio, escudo) y de jugadores (nombre, documento, dorsal, posición, foto). Valida que un documento no se repita. Genera los datos del carnet digital. Almacena escudos y fotos y solo guarda su dirección en la base.
- **Es dueño de:** equipos, jugadores, la relación delegado → equipos (un delegado puede tener uno o más).
- **No hace:** no sabe en qué grupo juega un equipo ni sus resultados (eso es de `matches-service`).
- **Regla de permisos:** el delegado solo modifica **su** equipo y **su** plantilla; los administradores de un torneo solo ven los equipos inscritos en sus torneos (ver 12.7); el público ve nombre y escudo.
- **Publica:** `equipo.creado`, `equipo.actualizado`, `equipo.eliminado`, `jugador.agregado`, `jugador.actualizado`, `jugador.eliminado`.
- **Consume:** `usuario.desactivado` (para marcar equipos sin delegado activo).

`matches-service` — competición

- **Responsabilidad:** los torneos en sí (cada torneo con sus administradores, sus equipos inscritos, sus grupos y qué equipo juega en cada grupo), generación automática de liguillas, programación de partidos (fecha, hora, cancha, fase), registro de resultados y de eventos del partido (goles, asistencias, tarjetas), **tabla de posiciones**, goleadores, valla menos vencida y exportación a Excel.
- **Es dueño de:** torneos, administradores de cada torneo, inscripciones de equipos, grupos, partidos, eventos de partido, y una copia mínima de equipos y jugadores (identificador, nombre, escudo, dorsal).
- **No hace:** no decide multas ni suspensiones; solo informa que hubo una tarjeta.
- **Publica:** `partido.programado`, `partido.reprogramado`, `partido.cancelado`, `resultado.registrado`, `gol.registrado`, `tarjeta.registrada`.
- **Consume:** `equipo.*` y `jugador.*` para mantener su copia mínima.

`discipline-service` — sanciones

- **Responsabilidad:** convertir tarjetas en consecuencias: multas económicas con su estado de pago, acumulación de amarillas, suspensiones automáticas y su cumplimiento. Guarda el reglamento (cuántas amarillas suspenden, valor de cada

multa).

- **Es dueño de:** multas, sanciones y suspensiones, parámetros del reglamento, y copia mínima de jugadores y equipos.
- **No hace:** no programa partidos ni modifica resultados.
- **Regla de permisos:** los administradores del torneo correspondiente gestionan y marcan pagos; el delegado solo consulta lo de su equipo.
- **Publica:** `multa.creada`, `multa.pagada`, `suspension.aplicada`, `suspension.cumplida`.
- **Consume:** `tarjeta.registrada`, `resultado.registrado` (para dar por cumplida una suspensión), `equipo.*` y `jugador.*`.

`notifications-service` — avisos y correos

- **Responsabilidad:** notificaciones dentro de la plataforma para el delegado (partido programado o cambiado, multa, suspensión) y envío de correos, incluida la recuperación de contraseña. Es el único que conoce las claves del proveedor de correo.
- **Es dueño de:** notificaciones (destinatario, tipo, mensaje, leída o no), plantillas de correo, registro de envíos, y copia mínima de usuario y equipo (para saber a quién y a qué correo avisar).
- **No hace:** no decide cuándo avisar; reacciona a eventos de los otros.
- **Publica:** nada.
- **Consume:** `usuario.*`, `equipo.*`, `partido.*`, `resultado.registrado`, `multa.*`, `suspension.*`.

12.4 Qué servicio hace cada función del informe

Función del informe GESTOR	Servicio responsable
Inicio de sesión, recuperación de contraseña, roles	`auth-service`
Registro de equipo con escudo y municipio	`auth-service` (cuenta) + `teams-service` (equipo)
Aceptar política de privacidad	`auth-service`
Gestionar plantilla de jugadores	`teams-service`
Carnet digital del jugador	`teams-service`
Crear grupos del torneo	`matches-service`
Programar partidos y generar liguillas	`matches-service`

Función del informe GESTOR	Servicio responsable
Registrar resultados y eventos del partido	`matches-service`
Tabla de posiciones y exportar a Excel	`matches-service`
Goleadores y valla menos vencida	`matches-service`
Tarjetas, multas y suspensiones automáticas	`discipline-service`
Notificaciones al delegado y correos	`notifications-service`
Vista pública de fixture y posiciones, tiempo real	`api-gateway` (sirve) + `matches-service` (datos)
Dashboard del administrador (conteos y estadísticas)	`api-gateway` reúne datos de `teams`, `matches` y `discipline`

12.5 Mapa de eventos

Evento	Lo publica	Lo consumen
`usuario.registrado`, `usuario.correo_actualizado`	`auth`	`notifications`
`usuario.recuperacion_solicitada`	`auth`	`notifications`
`usuario.desactivado`	`auth`	`teams`, `notifications`
`equipo.creado`, `equipo.actualizado`, `equipo.eliminado`	`teams`	`matches`, `discipline`, `notifications`
`jugador.agregado`, `jugador.actualizado`, `jugador.eliminado`	`teams`	`matches`, `discipline`
`partido.programado`, `partido.reprogramado`, `partido.cancelado`	`matches`	`notifications`
`resultado.registrado`	`matches`	`discipline`, `notifications`
`gol.registrado`	`matches`	(reservado para estadísticas futuras)
`tarjeta.registrada`	`matches`	`discipline`
`multa.creada`, `multa.pagada`	`discipline`	`notifications`

Evento	Lo publica	Lo consumen
`suspension.aplicada`, `suspension.cumplida`	`discipline`	`notifications`

Ejemplo completo: el admin registra un resultado con una tarjeta roja → `matches` guarda y publica `resultado.registrado` y `tarjeta.registrada` → `discipline` crea la multa y la suspensión y publica `multa.creada` y `suspension.aplicada` → `notifications` avisa al delegado dentro de la plataforma y por correo. Si el correo falla, el resultado ya quedó guardado.

12.6 Decisiones tomadas y por confirmar

Tomadas en este catálogo

- Los grupos y la pertenencia de equipos a grupos pertenecen a `matches-service`, no a `teams-service`, porque son parte de la estructura del torneo.
- Cero llamadas entre servicios; solo copias mínimas por eventos.
- El correo lo envía siempre `notifications-service`.
- El frontend es una aplicación aparte, no un microservicio.

Confirmado por ti

- Se trabaja con **5 servicios más el gateway**, cada uno con su propia base de datos.
- La regla del delegado: gestiona solo la plantilla de su equipo y lee lo demás.
- Varios administradores por torneo, todos iguales, y varios torneos por administrador (ver 12.7).
- Se parte de cero con las bases de datos (ver sección 13).

12.7 Aclaración: varios torneos y varios administradores (prevalece sobre lo anterior)

Qué cambia. El torneo deja de ser algo implícito y pasa a ser una entidad principal. Un torneo tiene **uno o varios administradores**, y una persona administradora puede tener **uno o varios torneos**. Es una relación de muchos a muchos. Esto reemplaza la idea de una "temporada" única y cualquier mención de un solo administrador en las secciones anteriores.

Dos niveles de rol

Nivel	Rol	Qué puede hacer	Dónde se guarda
Cuenta	`admin`	Crear torneos y ser administrador de ellos.	`auth-service`
Cuenta	`delegado`	Gestionar su equipo y su plantilla; inscribir su equipo en torneos.	`auth-service`
Dentro de un torneo	administrador	Todo lo del torneo, sin jerarquías: gestionar equipos inscritos, grupos, partidos, resultados y sanciones; agregar o retirar a otros administradores; archivar el torneo. Quien crea el torneo queda como un administrador más, igual a los demás.	`matches-service`

Tener el rol `admin` en la cuenta **no basta** para tocar un torneo: hay que ser administrador de ese torneo en concreto.

Todos los administradores de un torneo son iguales. Como cualquiera puede retirar a otro, se agregan dos protecciones: un torneo **siempre conserva al menos un administrador** (nadie puede retirar al último ni retirarse si es el único), y toda alta o baja de administradores queda en un **registro de auditoría** con quién lo hizo y cuándo.

Qué cambia en cada servicio

Servicio	Cambio
`auth-service`	Guarda y valida los **códigos de acceso** con los que se crean las cuentas `admin`. El rol `admin` significa "puede ser organizador".
`matches-service`	Es dueño de: **torneos**, **administradores de cada torneo**, **inscripciones** (qué equipo participa en qué torneo y en qué grupo) y la **plantilla habilitada por torneo** (donde se garantiza que un documento no se repita dentro del torneo). Guarda una copia mínima de las cuentas `admin` para poder agregarlas a un torneo por correo. Grupos, partidos, eventos y posiciones pertenecen a un torneo.

Servicio	Cambio
`teams-service`	Un equipo es una identidad independiente del torneo y puede inscribirse en varios. Ya no lo elimina un administrador de torneo; un administrador solo **retira la inscripción** de su torneo (eso es de `matches-service`). Guarda una copia mínima de "qué equipos están inscritos en qué torneo" y "quién administra cada torneo" para decidir quién puede ver una plantilla.
`discipline-service`	Multas, sanciones y suspensiones llevan el identificador del torneo; las suspensiones valen dentro del torneo donde ocurrieron. Autoriza con su copia de administradores por torneo.
`notifications-service`	Los avisos indican a qué torneo pertenecen. Los administradores de un torneo reciben los avisos de ese torneo, por ejemplo una solicitud de inscripción nueva.
`api-gateway`	Las rutas públicas pasan a ser por torneo, por ejemplo `/api/public/torneos/{id}/posiciones`. El dashboard del administrador se arma por torneo.

Cómo se decide un permiso sin llamar a otro servicio. `matches-service` publica `torneo.administrador_agregado` y `torneo.administrador_retirado`; `teams`, `discipline` y `notifications` guardan una copia mínima de esa lista y la consultan en local. Así se mantiene la regla de cero llamadas entre servicios.

Eventos nuevos

Evento	Lo publica	Lo consumen
`usuario.registrado`, `usuario.correo_actualizado` (de cuentas `admin`)	`auth`	`matches` (además de `notifications`)
`torneo.creado`, `torneo.archivado`	`matches`	`discipline`, `notifications`
`torneo.administrador_agregado`, `torneo.administrador_retirado`	`matches`	`teams`, `discipline`, `notifications`
`inscripcion.solicitada`	`matches`	`notifications` (avisa a los administradores)

Evento	Lo publica	Lo consumen
`inscripcion.aceptada`, `inscripcion.rechazada`, `inscripcion.retirada`	`matches`	`teams`, `discipline`, `notifications`
`jugador.en_conflicto` (documento repetido en el torneo)	`matches`	`notifications` (avisa al delegado y a los administradores)

Flujos principales

1. **Crear una cuenta de administrador:** el desarrollador genera un **código de acceso** (de un solo uso, con vencimiento y, si se quiere, atado a un correo) y se lo entrega a la persona junto con el enlace de la página de registro de administradores. La persona crea su cuenta ingresando ese código, y el código queda consumido. No existe auto-registro de administradores.
2. **Crear un torneo:** una cuenta `admin` lo crea en `matches-service` y queda como administrador de ese torneo.
3. **Agregar otro administrador a un torneo:** un administrador del torneo elige, por correo, a una persona que **ya tenga cuenta `admin`** y se agrega directamente. Si todavía no tiene cuenta, debe pedirle un código de acceso al desarrollador.
4. **Inscribir un equipo:** el delegado solicita unirse a un torneo desde su equipo y la inscripción queda `pendiente`. Un administrador del torneo la **aprueba o la rechaza** (con motivo). Con la aprobación se publica `inscripcion.aceptada`.

Impacto en el informe del proyecto. La limitación 4.2 dice que el sistema no soporta varios torneos simultáneos desde una misma cuenta de administrador. Con esta aclaración esa limitación deja de ser cierta y conviene actualizar el alcance del informe. El sistema sigue siendo solo para fútbol.

Decisiones cerradas

1. **Cuenta de administrador:** solo se crea con un código de acceso que entrega el desarrollador. No hay auto-registro.
2. **Inscripción de equipos:** siempre requiere la aprobación de un administrador del torneo.
3. **Un equipo en varios torneos:** permitido, a la vez.
4. **Documento de identidad:** no puede repetirse dentro de un mismo torneo, pero sí puede aparecer en torneos distintos.

13. Diseño de las bases de datos (desde cero)

Definición de las 5 bases de datos, una por servicio. Es diseño, no código: cada tabla se describe con sus campos y reglas. Los nombres van en español y en `snake_case`, como en el sistema actual.

13.1 Principios comunes

1. **Una base por servicio y un usuario de base de datos por servicio**, con permisos solo sobre su propia base.
2. **Ninguna clave foránea entre bases.** Una referencia a otro servicio se guarda como un identificador (UUID) sin llave. Dentro de una misma base sí hay claves foráneas.
3. **Identificadores UUID** generados por la aplicación.
4. **Fechas y horas en UTC**, mostradas en hora de Colombia en el frontend.
5. **Dinero en pesos colombianos** como número decimal exacto, nunca como decimal flotante.
6. **Borrado lógico** (campo `eliminado_en`) para lo que tiene historial: equipos, jugadores, torneos. Los códigos y tokens vencidos sí se borran de verdad.
7. **Tablas `copia_*`:** copias mínimas de datos de otros servicios. El servicio nunca las modifica por su cuenta; solo cambian al recibir eventos.
8. **Dos tablas técnicas en toda base:** `eventos_salida` (eventos pendientes de publicar, para no perderlos) y `eventos_procesados` (identificadores de eventos ya aplicados, para que un evento repetido no se aplique dos veces).
9. **Auditoría** de las acciones administrativas: quién, qué, cuándo, desde dónde (la IP se guarda en forma de hash).
10. **Migraciones versionadas** que solo avanzan; nunca se edita una migración ya aplicada.

13.2 `auth_db` — `auth-service`

Tabla	Campos principales	Reglas
`usuarios`	id, correo, nombre, contrasena_hash, rol (`admin` o `delegado`), estado (`activo`, `bloqueado`, `desactivado`), intentos_fallidos, bloqueado_hasta, creado_en, actualizado_en	Correo único sin distinguir mayúsculas. La contraseña se guarda con Argon2id, nunca en claro.

Tabla	Campos principales	Reglas
`sesiones`	id, usuario_id, token_hash, familia_id, expira_en, revocado_en, reemplazado_por, agente_usuario, ip_hash, creado_en	Cada renovación crea un token nuevo de la misma familia. Si se reutiliza un token ya usado, se revoca toda la familia.
`recuperaciones_contrasena`	id, usuario_id, token_hash, expira_en, usado_en, creado_en	Un solo uso y vida corta. Solo se guarda el hash.
`codigos_acceso_admin`	id, codigo_hash, correo_destino (opcional), expira_en, usado_en, usado_por, creado_por, intentos_fallidos, creado_en	Un solo uso. Solo se guarda el hash. Si tiene `correo_destino`, solo sirve para ese correo. Se bloquea tras varios intentos fallidos.
`consentimientos`	id, usuario_id, version_politica, aceptado_en, ip_hash	Se registra al crear la cuenta y cuando cambia la política.
`auditoria_auth`	id, usuario_id (puede ser nulo), accion, detalle, ip_hash, creado_en	Inicios de sesión fallidos, cambios de contraseña, códigos generados y usados.

13.3 `teams_db` — `teams-service`

Tabla	Campos principales	Reglas
`equipos`	id, nombre, municipio, escudo_url, delegado_id, estado, creado_en, actualizado_en, eliminado_en	`delegado_id` es el id de usuario en `auth` (sin llave). Un delegado no puede repetir el nombre de dos equipos suyos.

Tabla	Campos principales	Reglas
`jugadores`	id, equipo_id, nombres, apellidos, tipo_documento, documento_cifrado, documento_hash, fecha_nacimiento, dorsal, posicion, foto_url, estado, creado_en, actualizado_en, eliminado_en	`documento_cifrado` se cifra en la aplicación. `documento_hash` es un hash con clave secreta que permite detectar repetidos sin descifrar. Dentro de un equipo no se repite ni el documento ni el dorsal.
`copia_torneos`	torneo_id, nombre, estado	Para mostrar en qué torneos participa un equipo.
`copia_inscripciones`	equipo_id, torneo_id, estado	Para decidir quién puede ver una plantilla.
`copia_administradores_torneo`	torneo_id, usuario_id	Para autorizar a los administradores de un torneo.

13.4 `matches_db` — `matches-service`

Tabla	Campos principales	Reglas
`torneos`	id, nombre, descripcion, temporada, formato (`grupos` o `liguilla`), estado (`borrador` , `inscripciones_abiertas` , `en_juego` , `finalizado` , `archivado`), fecha_inicio, fecha_fin, cupo_maximo_equipos, puntos_victoria, puntos_empate, puntos_derrota, criterios_desempate, creado_por, creado_en, archivado_en	Los puntos y el desempate se definen por torneo.
`administradores_torneo`	torneo_id, usuario_id, agregado_por, agregado_en, retirado_en	Clave (torneo, usuario). Un torneo siempre conserva al menos un administrador activo.

Tabla	Campos principales	Reglas
`inscripciones`	id, torneo_id, equipo_id, solicitada_por, estado (`pendiente`, `aceptada`, `rechazada`, `retirada`), motivo_decision, decidida_por, decidida_en, creado_en	Solo puede haber una inscripción `pendiente` o `aceptada` por equipo en cada torneo. Un equipo puede tener inscripciones en varios torneos.
`grupos`	id, torneo_id, nombre, orden	Pertenecen a un torneo.
`grupo_equipos`	grupo_id, inscripcion_id	Una inscripción está en un solo grupo.
`plantillas_torneo`	id, torneo_id, inscripcion_id, jugador_id, documento_hash, estado (`habilitado`, `en_conflicto`, `retirado`)	**Un `documento_hash` no puede estar habilitado dos veces en el mismo torneo.** Es la regla del documento único por torneo.
`canchas`	id, torneo_id, nombre, direccion	Opcional; si no, la cancha es un texto del partido.
`partidos`	id, torneo_id, fase, jornada, grupo_id (opcional), inscripcion_local_id, inscripcion_visitante_id, fecha_hora, cancha_id, estado (`programado`, `en_juego`, `finalizado`, `aplazado`, `cancelado`), goles_local, goles_visitante, registrado_por, creado_en, actualizado_en	Local y visitante deben ser distintos y del mismo torneo.
`eventos_partido`	id, partido_id, tipo (`gol`, `autogol`, `asistencia`, `amarilla`, `roja`), jugador_id, inscripcion_id, minuto, registrado_por, creado_en	Alimenta goleadores, asistencias y disciplina.
`posiciones`	torneo_id, grupo_id, inscripcion_id, pj, pg, pe, pp, gf, gc, dg, puntos, actualizado_en	Se recalcula en la misma transacción que guarda un resultado, así nunca queda desactualizada.

Tabla	Campos principales	Reglas
`copia_usuarios_admin`	usuario_id, correo, nombre, activo	Para agregar administradores por correo.
`copia_equipos`	equipo_id, nombre, municipio, escudo_url, delegado_id, eliminado	Para mostrar equipos y verificar quién es su delegado al solicitar una inscripción.
`copia_jugadores`	jugador_id, equipo_id, nombre_completo, dorsal, posicion, documento_hash, activo	Para goleadores y para armar la plantilla del torneo.
`auditoria_torneos`	id, torneo_id, usuario_id, accion, detalle, creado_en	Altas y bajas de administradores, decisiones de inscripción, cambios de resultado.

13.5 `discipline_db` — `discipline-service`

Tabla	Campos principales	Reglas
`reglamentos`	id, torneo_id, amarillas_para_suspension, partidos_suspension_por_roja, valor_multa_amarilla, valor_multa_roja, vigente_desde	Uno vigente por torneo. Los valores los define el administrador del torneo.
`tarjetas`	id, torneo_id, partido_id, jugador_id, equipo_id, tipo (`amarilla`, `roja`), minuto, evento_origen_id, creado_en	`evento_origen_id` es único: si el mismo evento llega dos veces no se duplica la tarjeta.
`acumulados_jugador`	torneo_id, jugador_id, amarillas_acumuladas, actualizado_en	Clave (torneo, jugador). Se usa para decidir la suspensión por acumulación.

Tabla	Campos principales	Reglas
`multas`	id, torneo_id, equipo_id, jugador_id (opcional), tarjeta_id (opcional), concepto, monto, estado (`pendiente` , `pagada` , `anulada`), pagada_en, registrada_por, marcada_pagada_por, creado_en	Pueden ser automáticas (por tarjeta) o manuales del administrador.
`suspensiones`	id, torneo_id, jugador_id, motivo (`acumulacion` , `roja` , `manual`), partidos_totales, partidos_cumplidos, estado (`vigente` , `cumplida` , `anulada`), tarjeta_id (opcional), creado_en	Vale solo dentro del torneo donde ocurrió.
`copia_torneos` , `copia_administradores_torneo`	torneo_id, nombre, estado / torneo_id, usuario_id	Para autorizar a los administradores.
`copia_equipos` , `copia_jugadores`	equipo_id, nombre, delegado_id / jugador_id, equipo_id, nombre_completo	Para mostrar nombres y saber qué delegado ve qué multas.
`auditoria_disciplina`	id, torneo_id, usuario_id, accion, detalle, creado_en	Pagos marcados, multas anuladas, suspensiones modificadas.

13.6 `notifications_db` — `notifications-service`

Tabla	Campos principales	Reglas
`plantillas`	id, clave, asunto, cuerpo, version, activa	Textos de los correos, por ejemplo la recuperación de contraseña. `clave` es única por versión.

Tabla	Campos principales	Reglas
`notificaciones`	id, usuario_destino_id, equipo_id (opcional), torneo_id (opcional), tipo, titulo, mensaje, datos, leida_en, creado_en, expira_en	Las que ve el delegado o el administrador dentro de la plataforma.
`envios_correo`	id, notificacion_id (opcional), usuario_id, correo_destino, plantilla_clave, estado (`pendiente` , `enviado` , `fallido`), intentos, ultimo_error, proximo_intento_en, enviado_en, creado_en	Los fallos se reintentan; así un correo caído no afecta al resto del sistema.
`preferencias_usuario`	usuario_id, correo_activo	Permite que alguien pida no recibir correos de avisos. Los de seguridad se envían siempre.
`copia_usuarios`	usuario_id, correo, nombre, rol, activo	Para saber a quién y a qué correo avisar.
`copia_equipos` , `copia_torneos` , `copia_administradores_torneo`	equipo_id, nombre, delegado_id / torneo_id, nombre / torneo_id, usuario_id	Para decidir los destinatarios de cada aviso.

13.7 Reglas que cruzan servicios

Documento único por torneo. `teams-service` impide repetir un documento dentro de un mismo equipo. Para el torneo, la regla se garantiza en `matches-service` : al inscribirse un equipo, sus jugadores pasan a `plantillas_torneo` con su `documento_hash` , y la base impide dos jugadores habilitados con el mismo hash en un torneo. Si hay choque, el segundo queda `en_conflicto` (no habilitado) y se avisa al delegado y a los administradores. Como el aviso viaja por evento, hay una ventana de segundos en la que el conflicto aún no se ve; es aceptable porque el jugador no queda habilitado.

Inscripción. El delegado solo puede solicitar la inscripción de un equipo suyo; `matches-service` lo verifica con `delegado_id` de su `copia_equipos`. Tras un rechazo se puede volver a solicitar.

Eliminar un equipo. Es un borrado lógico. Los partidos y resultados pasados conservan el nombre y el historial.

Eventos repetidos o fuera de orden. Cada servicio aplica un evento una sola vez (`eventos_procesados`) y descarta el que llega con una versión más vieja que la copia que ya tiene.

13.8 Datos sensibles

Dato	Dónde	Protección
Contraseña	`auth_db`	Argon2id; nunca en claro ni en logs.
Tokens de sesión, recuperación y códigos de acceso	`auth_db`	Solo se guarda su hash.
Documento de identidad	`teams_db`	Cifrado en la aplicación; en las demás bases solo existe el hash. Nunca aparece en vistas públicas.
Fecha de nacimiento y foto	`teams_db`	Acceso solo al delegado del equipo y a administradores de torneos donde el equipo esté inscrito.
Correo	`auth_db`, copia en `notifications_db` y `matches_db` (solo admin)	Solo se copia lo necesario para avisar o para agregar administradores.
Dirección IP	`auth_db`	Se guarda como hash.

13.9 Dónde viven en local y en producción

- **Local:** un solo contenedor de PostgreSQL 16 con las 5 bases y 5 usuarios. Cada servicio recibe solo la cadena de conexión de su base.
- **Producción:** 5 proyectos independientes en Neon (uno por servicio), cada uno con su usuario de mínimos privilegios. El tamaño proyectado cabe de sobra en 1 GB por base.

13.10 Por confirmar de la parte de datos

1. **Criterios de desempate** de la tabla de posiciones (por ejemplo diferencia de gol, goles a favor, enfrentamiento directo). El informe no los define; propongo que cada torneo los configure como una lista ordenada.
2. **Jugadores menores de edad:** propongo guardar si existe autorización del acudiente, porque el sistema registra fecha de nacimiento y fotos.
3. **Retención:** propongo borrar tokens y códigos vencidos a los 30 días, notificaciones leídas a los 12 meses y conservar la auditoría 24 meses. Son valores para que los ajustes tú.